JAVA SCRIPT.

- JAVA SCRIPT IS A PROGRAMMING LANGUAGE, PROPOSED BY 'BRENDAN EICH' ON MAY 1995.
- THIS LANGUAGE WAS FIRSTLY USED / IMPLIMENTED BY 'Netscape Communication Corporation'.THEY NAMED THIS LANGUAGE AS 'MOCHA (INITIAL NAME OF JAVASCRIPT LANGUAGE)' (THE NAME MOCHA IS SUGGESTED BY 'MARC ANDREESSEN'(A NETSCAPE FOUNDER.)).
- 'MOCHA' IS OFTEN NAMED AS LIVESCRIPT ON 'SEPTEMBER 1995'.
- ON DECEMBER 1995 THIS NAME 'LIVE SCRIPT' GOT A COPYRIGHT ISSUE BY A ORGAINSATION CALLED 'SUN'(THEY OWN THE TREDMARK LICENCE ON THIS NAME). THEREFORE IT IS OFTEN NAMED AS 'JAVASCRIPT'.
- AN ORGANIZATION CALLED 'ECMA(EUROPEAN COMPUTER MANUFACTURES ASSOCIATION.) INTERNATIONAL' HAS OFTEN ADOPTED JAVASCRIPT AND DECLARED JAVASCRIPT AS A STANDARD PROGRAMMING LANGUAGE TO BE USED BY ALL BROWSER TO DEVELOPOPE WEB APPLICATIONS.
- ECMA USED TO LAUNCH THE UPGRADED VERSIONS OF JAVASCRIPT AS ECMA SCRIPT VERSIONS

HOW TO PERFORM JAVASCRIPT:-

- JAVA SCRIPT CAN BE PROGRAMMED IN 2 WAYS,
  1) INTERNAL JAVA-SCRIPT.
  2) EXTERNAL JAVA-SCRIPT.

1) INTERNAL J.S :-

- WE CAN WRITE THE INTERNAL J.S IN THE HEAD PART OR BODY PART OF HTML DOCUMENT.
- WE HAVE TO USE <script> TAG TO WRITE THE INTERNAL J.S. EXAMPLE:-

```
<head>
    <title>Document</title>
</head>
<body>
    <script>
        document.write("HELLO WORLD")
    </script>
</body>
```

2) EXTERNAL JAVA SCRIPT:-

- **IN EXTERNAL J.S, WE MUST HAVING 2 FILES.**
- **ONE IS HTML FILE, AND ANOTHER ONE IS J.S FILE.**
- **WE HAVE LINK HTML FILE WITH J.S FILE BY USING <script> TAG.**
- **IN <script> TAG WE HAVE TO USE SOURCE(src) ATTRIBUTE.**
- **IN SRC WE HAVE TO GIVE EXTERNAL J.S 'url' with 'extension'.**

Example:-

HTML FILE:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Document</title>
</head>
<body>
    <script src="./JS1.js">

    </script>
</body>
</html>
```

JAVA SCRIPT FILE:-

```
document.write("HELLO WORLD")
```

CHARACTERISTICS OF JAVASCRIPT:

- **IT IS A PROGRAMMING/SCRIPTING LANGUAGE.**
- **IT IS A DYNAMIC LANGUAGE**

- IT IS A SYNCHRONOUS LANGUAGE.
- SINGLE THREADED LANGUAGE.
- IT IS A HIGH LEVEL LANGUAGE.
- IT IS A INTERPRETED LANGUAGE.

JAVASCRIPT TOKEN:-

TOKEN is a Javascript statement CONSISTS OF 3 MAIN COMPONENTS.

I. VARIABLE.
II. IDENTIFIER.
III. VALUE.

EXAMPLE:-

let a =10

note:-

- we can consider any thing as a identifier including '$' and '_' special character.(no other special characters). And also we can not use any pre-defined javascript keywords as an identifier.
- when we use any vaiable with multiple words, then we must write the variable name followed by lowercase camelcasing without giving any spaces in between them.

EX:-

```
let dingaDon = "10"
console.log(dingaDon);
let $="hello"
console.log($)
let _="hello"
console.log(_)
let &="hello"
console.log(&)
// OTHER SPECIAL CHARACTERS OTHERTHAN $ AND _ ARE NOT ALLOWED TO BE TAKEN AS AN IDENTIFIER.
```

VARIABLES:-

- IN JS WE HAVE 3 VARIABLES THAT IS,
- var , let, const

VARIABLE DECLARATION & INITIALIZATION:-

Var a; (this is variable declaration)
a=10;(this is variable initialization)

var:-

- WHENEVER WE HAVE TO STORE THE VALUES IN A VARIABLE WHICH CAN BE RE-DECLARED AND RE-INITIALIZED WHENEVER REQUIRED, THEN WE HAVE TO TAKE var VARIABLE.
- EXAMPLE:-

```
var x
console.log(x)
// VD is possible.
let x
console.log(x)
// RD is possible
x=10
console.log(x)
// VI is possible
x="hello"
console.log(x)
// RI is possible
var x = "100"
console.log(x)
// VD & VI is possible
x="bye"
console.log(x)
// RD & RI is possible.
```

let:-

- WHENEVER WE HAVE TO STORE THE VALUES IN A VARIABLE WHICH SHOULD NOT BE RE-DECLARE BUT CAN BE RE-INITIALIZED WHENEVER REQUIRED, THEN WE HAVE TO TAKE let VARIABLE.
- EXAMPLE:-

```
let x
console.log(x)
// VD is possible
// var x
// console.log(x)
// RD is not possible
x=10
```

```
console.log(x)
// VI is possible
x="hello"
console.log(x)
// RI is possible
let x=100;
console.log(x)
// VD & VI are possible.
var x=200;
console.log(x);
// RD & RI is not possible.
```

Const:-

- **WHENEVER WE HAVE TO STORE THE VALUES IN A VARIABLE WHICH SHOULD NOT BE BE RE-DECLARED AND RE-INITIALIZED AGAIN IN THE OUR PROGRAM AGAIN, THEN WE HAVE TO TAKE const VARIABLE.**

```
const x
console.log(x)
// VD is not possible.
const x = "shakeela"
console.log(x)
// VD & VI is possible.
x=10
console.log(x)
// RD is not possible
```

VALUES IN JS:-

PRIMITIVE DATATYPE	NON-PRIMITIVE DATATYPE
NUMBER Note:-including 'NAN'(NOT A NUMBER) it is a special number, STRING, BOOLEAN, NULL, UNDEFINED.	FUNCTIONS, ARRAYS, OBJECTS.

DIFFERENT O/P METHODS IN JS:-

1. console.log()
2. log() is method that belongs to console object. It is used to print an output value in the browser console platform.
3. EX:- `console.log("HELLO")`

4. console.error()
5. error() is a method belongs to console object. It is used to print an error message in the browser console platform.
6. EX:- console.error(`"there is an error in the code"`)

7. `console.warn()`
8. warn() is a method belongs to console object. It is used to print an warning message in the browser console platform.
9. Ex:- `console.warn("BROWSER WARNS YOU FOR THIS ACTION")`

10. document.write():-
11. it is used to print something in the UI / webpage.
12. Ex:- document.write('hello');

13. document.writeln():-
14. It is used print something in the UI / webpages with nbsp.
15. Ex:- document.writeln('hello'); document.writeln('hi');

16. window.alert():-
17. this method is used to give a pop up notification to the end user in the ui.
18. Ex:- window.alert('visit this page only if you are 18+');

19. window.prompt():-
20. this method is used to get an input from the end user through a notification box.
21. Ex:- window.prompt('type your value in the below input field');

22. window.confirm():-
23. the method is used to give the enduser a confirmation notification for which the end user have to give an respond as ok or cancel.
24. Ex: window.confirm('do you want to agree and proceed??')

HISTORY OF JAVA-SCRIPT

-->JavaScript was created in May 1995, by Brendan Eich.
Eich worked at Netscape and implemented JavaScript for their web browser,Netscape Navigator.

-->The idea was that major interactive parts of the client-side web.

-->Initially, JavaScript's name changed several times:
- Its code name was Mocha.
- In the Netscape Navigator 2.0 betas (September 1995), it was called LiveScript.
- In Netscape Navigator 2.0 beta 3 (December 1995), it got its final name, JavaScript

-->The "ECMA" in "ECMAScript" comes from the organization that hosts the primary standard.The original name of that organization was ECMA, an acronym for European Computer Manufacturers Association.

--> NOTE:
JAVA SCRIPT and ECMA SCRIPT both are same.

<----------->

WHAT IS JAVASCRIPT?WHY JS?

--->js is used to convert static-page into dynamic-page.
--->JavaScript (js) is a light-weight object-based and object-oriented programming language which is used by several websites for scripting the webpages.
--->JavaScript is a client-side scripting language and one of the most efficient, commonly used scripting languages. The term .client-side scripting language means that it runs at the client-side (or on the client machine) inside the web-browsers

<----------->

Note:
---JS is H.L.L (one can easily read)
---Browser:it is an application which provides environment to run JS instructions from HLL to MLL.
---JS code can run both inside and outside the browser.
Inside browser - JS engine. Outside browser - Node.js

---JS engine:
--Node:it is the combination of c++ and V8(Chrome JS engine) with bundle of methods and rules.
--Node provides environment to run JS outside the browser.This invention helps the JS to gain its popularity in usage as a backend language.
--We can run JS both inside and outside the browser.

CHARACTERISTICS OF JAVASCRIPT:

1. Purely object based and object oriented programming language.
2. case sensitive
3. just-in-time compailer
4. Interpreted language
5. Synchronous(coz,it follows SINGLE threaded architecture which has only one stack)

<----------->

TO EXECUTE JS ON BROWSER:

1. we can execute js instructions directly in the console provided by the browser.
2. we can execute js instructions by embading it in html page.
   types: a.internal, b.external

a.internal:
with the help of script tag, we can embeded js instructions.
syntax:

```
html code ...
<script>

     js code...

</script>
```

b.external:
1. we can create a separate file for js with a extension .js
2. link the js file with html page using src attribute of script tag.

syntax:

```
html code ...
<script src="path/filename.js"> (if js file and html file in the
    same folder path is not required)

     js code...

</script>
html code ...
```

TOKEN: smallest unit of any programming language. And consists of

1. keywords:a pre defined reserved word which understandable by the js engine is known as keyword.

Note:every keyword must be in the lower case and is not used as identifiers.
Ex:if,for,let,const,while,do-while,null,switch

2. Identifiers:name given by the programmers to the components of js like variables,functions,class etc are known as identifiers.

--->Rules for identifiers:
---an identifier cann't start with number.
---except $ and _ no other special characters are allowed.
---we can not use keywords as idetifiers.

3.Literals/values:data which is used in the js program is called as literals/values.
types of values in js:
---->this are all primitive data types
1.Number
2.string
3.Boolean
4.Null
5.undefined
6.Bigint

---->non-premitive data types are:
object,array,class,functions,maps,sets etc

<----------->

Number
it represets mathamatical operations.
eg: 1 , -1 ,0.1 , -0.1 etc..

STRINGS:
in js strings can be represented using single quote,double quotes or backticks.
Note:
-The start and end of a string must be done with the help of same quote.
-If a text contains single quote then it can be represented using double quotes.
Ex:
text: 'I'm a doctor'
in js: "I'm a

Backtick:
A backtick can be multiline string.
Ex:
var hobbies = `My hobbies are:
1.Cricket
2.Movies
3.stock`;
console.log(hobbies);
Note:
-the strings represneted using backticks is also known as template
The advantage of it is we can substitute and express it using ${}.

BOOLEAN
it can hold two values true and false.
it is useful for cotrolling program flow like if,switch.

undefined

-->a variable that has not been assigned a value that is undefined.
-->the undefined property indicates that a variable has not been assigned value.

```
eg: var x;
    console.log(x)
    console.log(typeof x)
```

null
null represent absense of object value.

```
eg: var x = null;
     console.log(typeof x)
```

bigint

we can use bigint type to store numbers beyond the given range by suffixing the number with 'n'.

```
eg: var a = 120n;
    console.log(a)
```

<----------->

TYPEOF OPERATOR:-

1.it is a keyword used as an unary(checks only one data/value) operator to identify the type of data.

2.syntax: typeof value

ex:

```
console.log(typeof 1);//number
console.log(typeof 1n);//bigint
console.log(typeof '1');//string
console.log(typeof "1");//string
console.log(typeof `1`);//string
console.log(typeof true);//boolean
console.log(typeof false);//boolean
console.log(typeof False);//error.('f' should be in lower case)
console.log(typeof undefined);//undefined
console.log(typeof null);//object
```

Note:
typeof null is never null,it is considered as object

VARIABLES:

a named block of memory which is used to store a value is known as variables

(container to store data).

Note:

-in js variables are not strictly typed,it is dynamically typed.

Therefore it is not necessary to specify type

of data during variable declarations.

-in a variable we can store any type of value.

-it is mandatory to declare a variable before using.

-syntax: var/let/const identifier;

ex:

```
var a ;//declaration
a = 10;//initialization or assignment
console.log(a);//10
let b ;
b = 10;
console.log(b);//10
const c = 30 ;
console.log(c);//30
```

Note:

when a variable is declared in js,js engine implicitly assigns undefined

value to it.

ex:

```
var a;
console.log(a);//undefined
```

UNDERSTANDING EXECUTION IN JS:

1.everytime when js engine runs a js code,it will 1st create a

'global execution context'.

2.The Global Execution context has 2 parts:

a.variable area

b.functional/executional area

3.JS engine generally uses 2 phases to execute a JS code

--> phase 1:all the memory is allocated for the declarations in top to bottom order and assigned with the default value 'undefined' in variable area of global execution context.

-->phase 2:all the instructions get executed in the top to bottom order in execution area of global execution context.

HOISTING: JS allows a programmer to use a member (variable) before the declaration statement.This characteristic is known as Hoisting.

DIFFRENCES BETWEEN var let & const:

var:

1.var is global scoped

2.we can declare multiple variables with the same name(most recently created variable is used).

3.The value of variable can be modified.

4.Variables declared using var is hoisted,does not belong to temporal dead zone.Can be used before initialization.

Ex:console.log(a);//undefined

var(a);

Ex:

```
var a = 10;
a =20;
console.log(a);//20
```

let:

1.let is local scoped

2.we can not declare two variables with the same name within a block.

3.The value of variable can be modified.

4.Variables declared using let is hoisted,and it does belong to temporal dead zone.Can not be used before initialization.

Ex:console.log(a);//reference error(because of TEMPORAL DEAD ZONE)

var(a);

ex:

```
let a = 10;
a =20;
console.log(a);//20
```

const:

1.it is local scoped

2.we can not declare two variables with the name within a block.

3.The value of variable can not be modified.

4.Variables declared using let is hoisted,and it does belong to temporal dead zone.Can not be used before initialization.Ex:console.log(a);//reference error (because of TEMPORAL DEAD ZONE)

var(a);

ex:

```
const a = 10;
a =20;//typeError
```

console.log(a);//error,assign the value only once.

TEMPORAL DEAD ZONE:

The time between the variables which are created and its intialization is known as TDZ.

-in TDZ period we can not use variables.

-let and const belong to TDZ

Ex:

```
let a;
clg(a);//undefined
a=10;
```

-the problem occurs only if the variables used before declaration.

```
clg(a);//reference error (TDZ)
let a;
a=10;
```

---

type-casting:

The process of converting one type of data to another type of data.

implict type-casting

the process of converting one type of data into another type of data by JS engine implicitly(implicit typecasting),when a wrong type of data is used in the expression is known as type coercion(or implicit type casting).

EX:

console.log(10+'20');//1020,number is coverted to string

console.log(10-'2');//8,string is coverted to number

console.log(5-'a');//NaN, the string does not a have valid numeric value,

hence it is converted to a special number 'NaN'.

What is Nan?

NaN(Not a Number) is number!!!

What is Nan?

NaN(Not a Number) is number!!!

Any arithmetic operation with NaN, result is NaN

EXPLICIT TYPECASTING:

The process of converting one type of value to another type of value is

known as explicit typecasting.

Conversion of any number to number:

1.String to number:

if a string is valid number,we get the real number.

if the string consist any other character,then we get NaN as output.

EX:

console.log(typeof 'a');

console.log(Number('a'))//NaN

console.log('123'+10);//12310

console.log(Number('123')+10);//133,string '123' is converted to number explicitly(by programmer)

console.log(Number('123a'+10));//NaN

<----------→

FUNCTIONAL PROGRAMMING:

Passing a function as an argument to another function is known as functional programming.

Advantage of functional programming:

-Instead of creating a function with a specific task, functional programming helps programmer to generate function which can perform a generic task by accepting a function as a parameter.

ex:

```
function operation(a,b,task){
 let res = task(a,b);
 return res;
}
let res1 = operation(10,20,function(a,b){
 return a+b;
});
let res2 = operation(30,20,function(a,b){
 return a-b;
});
let res3 = operation(10,5,function(a,b){
 return a/b;
});
console.log(res1);
```

console.log(res2);

console.log(res3);

-In the above example the function operation which can accept a function(task) as a parameter is known as higher order function(operation()).

-The function operation which is passed as an argument to another function is known as callback function.

Hihger order function:A function which accepts function as a parameter is called as HO function.

callback function:the function which is passed as an argument is called call-back function

<----------→

FUNCTION:

-named block of instructions which is used to perform a specific task.

-Funtion gets executed only when it is called.

-the main advantage of function is ,we can achieve code reuseabilty.

-MAIN PURPOSE : code reuseabilty,which means-code once declared can be anywhere.

Note:

-In JS function are beautifull, every function is nothing but an object.

Syntax to create a function:

Generally we can create a function in 2 ways:

1.using function declaration statement(function statement)

2.function expression.

1.function declaration statement:

```
function identifier(pameter1,parameter2,...)
{
 statements
}
```

Note:

-function is an object

-name of a function is a variable which holds the reference of function object.

-creating a function using function statement supports function hoisting. therefore we can also a function before function declaration.

ex:

```
console.log('start');
console.log(test);
function test()
{
 console.log("hello")
}
console.log("end");
```

-When we try to log the function name the entire function definition is printed

PARAMETERS:

The variables declared in the function definition/declaration is known as parameter.

The parameters have local scope.Those can be used inside the function body

PARAMETERS are used to hold the values passed by a calling a function.

EX:

```
function sum(a,b)
```

```
{
 console.log(a+b);
}
```

here a&b are variables local to the function sum.

ARGUMENTS:

-The values passed in the method call statement is known as arguments.

-An argument can be a literal,variable or an expression which results a value

Ex:

sum(10,20)----- 10,20 are literals used as arguments

-ex2:

sum(-10+3,-20);//-27

-ex3:

let a=20,b=30

sum(a,b);a&b are variables used as arguments

RETURN:

-It is a keyword used as control transfer statement in a function.

-Return will stop the execution of the function and transfers the control along with data to the caller.

2.FUNCTION EXPRESSION:

SYNTAX:

var/let/const identifier = function(){}

in this syntax the function is used as value.

disadvantage:

the function is not hoisted, we can not use the function before declaration.

Reason:function is not hoisted,instead variable is hoisted and assigned with default value undefined.Therefore typeof a is not a function,it is undefined. Refer below ex,

```
a();//error; 'a' is not a function
var a = function()
 {
clg("hi");
}
clg(a):://function state or body is printed
a();//hi,function calling
a = 10;
a();//error, 'a' is not a function, coz, a is changed to number type from
function. This we call it as
Dynamic type. I can change variable during run time.
```

Note:

1.Any member declared inside a function will have local scope

Local scope:

The scope within the function block is known as local scope.

Any member with local scope can not be used outside the function block.

Ex:

1.A parameter of a function will have local scope

ex:function test(a){} ----------> here 'a' is variable whose scope is local to test and it can be used only inside test function block.

ex2:function test(){

var a; ----->it is local to test function,it can not be used outside.

}

ex3. function test(){

function insideTest()

{

}

}

Note:insideTest is function,local to test function and can not be called from outside test()

NOTE:

-Inside a function we can always use the members of global scope

Ex:

```
var city = 'Banglore';
function display(name){
 console.log(`${name} belongs to ${city}`;
}
display('Sheela');
```

in the above ex variable name has local scope and var city has global scope. It is observed inside function body we can use both the variables name & city.

this:

-It is a keyword used as variable.

-It holds the address of global window object.

-Therefore with the help of this variable,we can use members of global window object.

Ex:

```
var b = 20
function test(){
 var b = 30;
 console.log('from test '+b)//30
console.log(this.b);//20
}
test();
```

-In JS 'this' is a property of every function(every function will have 'this' keyword);

-Generally 'this' contains the address of current execution context to which the function belongs to.

<----------→

FUNCTION:

-named block of instructions which is used to perform a specific task.

-Funtion gets executed only when it is called.

-the main advantage of function is ,we can achieve code reuseabilty.

-MAIN PURPOSE : code reuseabilty,which means-code once declared can be anywhere.

Note:

-In JS function are beautifull, every function is nothing but an object.

Syntax to create a function:

Generally we can create a function in 2 ways:

1.using function declaration statement(function statement)

2.function expression.

1.function declaration statement:

```
function identifier(pameter1,parameter2,...)
{
 statements
}
```

Note:

-function is an object

-name of a function is a variable which holds the reference of function object.

-creating a function using function statement supports function hoisting. therefore we can also a function before function declaration.

ex:

```
console.log('start');
console.log(test);
function test()
{
 console.log("hello")
}
console.log("end");
```

-When we try to log the function name the entire function definition is printed

PARAMETERS:

The variables declared in the function definition/declaration is known as parameter.

The parameters have local scope.Those can be used inside the function body

PARAMETERS are used to hold the values passed by a calling a function.

EX:

```
function sum(a,b)
{
 console.log(a+b);
}
```

here a&b are variables local to the function sum.

ARGUMENTS:

-The values passed in the method call statement is known as arguments.

-An argument can be a literal,variable or an expression which results a value

Ex:

sum(10,20)----- 10,20 are literals used as arguments

-ex2:

sum(-10+3,-20);//-27

-ex3:

let a=20,b=30

sum(a,b);a&b are variables used as arguments

RETURN:

-It is a keyword used as control transfer statement in a function.

-Return will stop the execution of the function and transfers the control along with data to the caller.

2.FUNCTION EXPRESSION:

SYNTAX:

var/let/const identifier = function(){}

in this syntax the function is used as value.

disadvantage:

the function is not hoisted, we can not use the function before declaration.

Reason:function is not hoisted,instead variable is hoisted and assigned with default value

undefined.Therefore typeof a is not a function,it is undefined. Refer below ex,

```
a();//error; 'a' is not a function
var a = function()
 {
clg("hi");
}
clg(a)://function state or body is printed
a();//hi,function calling
a = 10;
a();//error, 'a' is not a function, coz, a is changed to number type from function. This we call it as
Dynamic type. I can change variable during run time.
```

Note:

1.Any member declared inside a function will have local scope

Local scope:

The scope within the function block is known as local scope.

Any member with local scope can not be used outside the function block.

Ex:

1.A parameter of a function will have local scope

ex:function test(a){} ----------> here 'a' is variable whose scope is local

to test and it can be used only

inside test function block.

ex2:function test(){

var a; ----->it is local to test function,it can not be used outside.

}

ex3. function test(){

function insideTest()

{

}

}

Note:insideTest is function,local to test function and can not be called

from outside test()

NOTE:

-Inside a function we can always use the members of global scope

Ex:

```
var city = 'Banglore';
function display(name){
 console.log(`${name} belongs to ${city}`;
}
display('Sheela');
```

in the above ex variable name has local scope and var city has global scope.

It is observed inside function body we can use both the variables name & city.

this:

-It is a keyword used as variable.

-It holds the address of global window object.

-Therefore with the help of this variable,we can use members of global window object.

Ex:

```
var b = 20
function test(){
 var b = 30;
 console.log('from test '+b)//30
console.log(this.b);//20
}
test();
```

-In JS 'this' is a property of every function(every function will have 'this' keyword);

-Generally 'this' contains the address of current execution context to which the function belongs to.

<----------→

ARROW FUNCTIONS:

-Arrow function was introduced from ES-6 version of javascript.

-main purpose is to reduce the syntax

-Syntax:

(parameter list,...) => {}

-Arrow functions can have 2 types of return:

a)implicit return:using 'return' keyword is not required.

syntax:

(parameters_list....)=>expressions

b)explicit return:using 'return' keyword is mandatory with curly brackets{},

if 'return' keyword is not used 'undefined' is given back.

-If a block is created an arrow function behaves like explicit return

syntax:

(parameters_list....)=>{return expressions}

eg:-

```
// const sum = (a,b) => a+b;
// console.log(sum(10,20))

// const product = (a,b) => a*b;
// console.log(product(10,20))
```

```
// const remaider = (a,b) => a % b;
// console.log(remaider(10,20))

// const difference = (a,b) => a-b;
// console.log(difference(10,20))
```

IMMIDIATE INVOKE FUNCTION EXPRESSION:

-When a function is called immediatly as soon as the function object is created it is known as Immidiate invokation.

-onetime use and throw

-Steps to achive IIFE:

a.Treat a function like a expression by declaring inside a pair of brackets.

b.Add another pair of brackets next to it which behaves like a function call statement.

eg:

```
let a = ( () => {console.log("hii")
            }) ();
```

NESTED FUNCTION:

-In JAVASCRIPT we can define a function inside another function.

-function outer(){

function inner(){

}

}

-The outer () is known as parent,and the inner() is known as child

Note:

-The inner() is local to outer function,it can not be accessed from the outside.

-To use inner function outside,the outer() must return the reference of inner().

Ex:

```
function outer(){
 function inner(){

 }
return inner ;
}

outer()()
```

-We can now call inner() from outside as follows:

-->type 1.

```
let fun = outer();
fun();
```

-->type 2.

```
outer ()();
```

SCOPE CHAIN(LEXICAL SCOPE):

-The ability of the javascript engine to search for a variable in the outer scope when it is not available in local scope is known as Scope chain.

-The scope chain is generally established between,

1.function & global object,

eg:-

```
let a = 10;
function test(){
    ++a
  console.log(a)
 }
```

2.child function & parent function with the help of closure.

2.child function & parent:(nested function)

ex:

```
let a =10;
function x(){
let b=20;
 function y(){
 console.log(b);
 console.log(a);
```

```
 }
 return y;
}
x()()
```

NOTE:

when the function y() is executed console.log(b) encountered,js engine looks for b in the local scope of function y(),since b is not present the y() is child of x() js engine will search for b in the parent function x() scope with the help of CLOSURE.

-We can achive scope chaining with the help of closure.

CLOSURE:

-The closure is a binding of parent function variables with the child function. This enables a JS programmer to use parent functions member inside child function.

<----------→

TO OBTAIN THE ELEMENTS FROM DOM:

methods of dom

1.getElementById():

-for this method we need to pass the ID of an element as a string ,it returns the first element specified ID.

Syntax:

```
document.getElementById("ID");
```

2.To fetch an element using 'name' attribute:

getElementsByName():

-we need to pass the name of an element as a string.

-It returns a nodelist,which is similar to array but not an array

-If there is no an element with the given name,then it returns empty nodelist

EX:(refer below ex in VS code)

```
let e1 = document.getElementsByName("heading")
console.log(e1[0]);
console.log(e1[0].textContent);
e1[0].textContent='welcome';
console.log(e1[0].textContent);
```

3.To fetch an element from DOM using the class name:

getElementsByClassName():

-Pass the class name as a string

-It returns a nodelist,containig all the elements matching the given html collection, which is similar to array but not an array

Syntax:

```
document.getElementByClass("class");
```

```
let e3 = document.getElementsByClassName("divstyle1")
console.log(e3[0].textContent);
```

4.To fetch the elements from the DOM using tag name

getElementsByTagName():

-We need to pass tag name as a string

-It returns an html collection containing all the elements matching the given tag name.

To fetch the elements from the DOM using CSS selectors:

-We can select an element from DOM using css selectors with the help of

1.querySelectorAll

2.querySelector

Note:

-We need to pass a css selectors as a string

-Query selector returns the reference of first element found.

-QuerySelectorAll returns a nodelist of all the elements found in the original order.

←----------*----------→

ARRAYS:

-Array is a huge block of memory which is used to store multiple values of different type.

--Arrays in JavaScript are heterogeneous. You can store any kind of data in the array.

--eg:-var arr = [2, "Hello", 123.5, true, person];

-To create an array :

a.(in javascript array is an object)

b.We can create array object in 2 different ways :

i)using array literal []

ii)by creating an instance of array,using new operator.

i)using array literal []:

Syntax:

let arr = [value1,value2,value3,.....] --->here 'arr' is the variable which stores the address of array object

EX:

```
let arr = [10,20,30];
console.log(arr)
```

----

ii)by creating an instance of array,using new operator.

Syntax:

let arr2 = new Array(); ---here '()' is function call

---> 'Array()' array function constructor

Array ----> anything starts with capital A it is class

() ---->constructor / constructor function which is having same name as class.

---> 'new' is keyword,which creates new object

note:an array object is creted but no data is

JavaScript array constructor (new keyword)

eg:

```
let arr3 = new Array(10,20,30);
```

creating a array objects and passing data to array and address of array of object is given to arr3.

To access the array elements :

-we can access array elements with the help of array object reference,array operator & index

syntax:

array_obj_ref[index]

Index: it is a number starts with zero and ends with the (length of array object - 1)

Ex:

```
let hobbies = ['reading','writing','playing cricket']
console.log(hobbies[1]);//writing
```

ADDING ELEMENTS INTO THE ARRAY OBJECT:

1.To add the element at the tail(last)

syntax:

array_ref[array_ref.length]=element;

ex:

```
let a = [10,20,30];
a[a.length]=40;//40 is added in the last
console.log(a);//[10,20,30,40]
```

Array.push method:

-it is a built in function of array object

-push function/method adds given arguments into the array object at last.

```
eg:let a = [10,20,30];
  a.push(5);
  clg(a);//[5,10,20,30]
```

note:

we can add an element into array using

syntax:

array_ref[index] = element;

```
eg:let a = [10,20,30];
   a[3]=50;
  clg(a);//[10,20,30,50]
```

---

to remove an elemnt from an array

pop(): to remove an element from tail of the array object

eg:

```
 let a = [10,20,30];
clg(a.pop());//30
```

----

array.shift()

The shift() method removes the first array element

eg:

```
  const a = [10,20,30,40,50];
  a.shift();//[20,30,40,50]
```

----

array.unshift()

The unshift() method adds a new element to an array (at the beginning)

or first index.

eg:

```
 const a = [10,20,30,40,50];
  a.unshift(1,5);//[1,5,10,20,30,40,50]
```

----

indexof()

the indexof() method it returns the index of particular elemet which is passed through it.

```
var arr = [10,210,30]
arr.indexof(20);//2
```

---

includes()

checks if an array contains a specified element

eg:

```
let languages = ["JavaScript", "Java", "C", "C++", "Python", "Lua"];

let check = languages.includes("Java");
console.log(check); // true
```

Array splice()

The splice() method returns an array by changing (adding/removing) its elements in place.

syntax:

```
arr.splice(start, deleteCount, item1, ..., itemN)
```

eg:

```
let prime_numbers = [2, 3, 5, 7, 9, 11];
let removedElement = prime_numbers.splice(4, 1, 13);
console.log(removedElement);

//Output: [ 9 ]
//        [ 2, 3, 5, 7, 13, 11 ]
```

Array slice()

---The slice() method returns selected elements in an array, as a new array.

--- creates a new array

syntax:

```
slice(start index,end index)
```

eg:

```
var num = [10,20,30,40]
num.slice(2,3);//[30]
```

<----------→

Array methods:

arr.push(...items) – adds items to the end,

arr.pop() – extracts an item from the end,

arr.shift() – extracts an item from the beginning,

arr.unshift(...items) – adds items to the beginning.

slice

The syntax is:

arr.slice([start], [end])

It returns a new array copying to it all items from index start to end (not including end).

Both start and end can be negative, in that case position from array end is assumed.

It's similar to a string method str.slice, but instead of substrings it makes subarrays.

```
let arr = ["t", "e", "s", "t"];

alert( arr.slice(1, 3) ); // e,s (copy from 1 to 3)

alert( arr.slice(-2) ); // s,t (copy from -2 till the end)
```

indexOf/lastIndexOf and includes

The methods arr.indexOf and arr.includes have the similar syntax and do essentially the same as their

string counterparts, but operate on items instead of characters:

arr.indexOf(item, from) – looks for item starting from index from, and returns the index where it was found, otherwise -1.

arr.includes(item, from) – looks for item starting from index from, returns true if found.

```
let arr = [1, 0, false];

alert( arr.indexOf(0) ); // 1
alert( arr.indexOf(false) ); // 2
alert( arr.indexOf(null) ); // -1

alert( arr.includes(1) ); // true
```

sort(fn)

The call to arr.sort() sorts the array in place, changing its element order.

It also returns the sorted array, but the returned value is usually ignored, as arr itself is modified.

For instance:

```
let arr = [ 1, 2, 15 ];

// the method reorders the content of arr
arr.sort();

alert( arr );  // 1, 15, 2
```

reverse

The method arr.reverse reverses the order of elements in arr.

For instance:

```
let arr = [1, 2, 3, 4, 5];
arr.reverse();

alert( arr ); // 5,4,3,2,1
```

Array.isArray

Array.isArray(value). It returns true if the value is an array, and false otherwise.

<----------*----------→

ASYNCHRONOUS

-The behaviour of making way for others is known as Asynchronous.

Explanation:

-Let us consider 2 functionalities f1&f2 purely independent from each other. Let us assume f1 takes 10 minutes to complete the execution,f2 takes just a minute to complete the execution.Therefore,if f1() is called first and then f2().

-f2() should wait for 10 minutes to get execution started. This behavious is synchronous.

-To overcome this we can design our application in such a way that f1() gives

way for f2() to complete its execution & then f1() can complete its execution.Such a design is known as Asynchronous.

-We can Asynchronous behaviour with the help of setTimeOut() method.

setTimeOut():

-It is a method of window object

-It accepts 2 arguments, argument 1 --> A callback function

argument 2 --> delay time in miliseconds

-It will register the given callback function and starts the browser timer for the given miliseconds when it is called.

-Once the set time is completed a callback function is moved to callback que.

-The 'event loop' loads the callback function from the callback que into the stack when the stack/callStack/mainStack is 'idol'

-Event loop waits untill callStack/mainStack become idol.

-Callstack becomes idol only after all the instructions of Global Execution Context is completed.

PROMISE:

Promise is an object

-Promise object keeps an eye on the asynchronous task given .

-If the asynchronous task not yet completed,the promise is considered as 'pending'.

-If the asynchronous task is successfully completed,then the promise is considered as 'resolved'.

-If the asynchronous task is completed but not successfull,then it is considered as 'reject'

SYNTAX TO CREATE A PROMISE OBJECT:

```
new Promise((resolve,reject)=>{

 asynchronous_task
} )
```

1.then():

-It can accept a callback function

-The callback function passed(given) to the then() methods gets executed only when the promise returns 'resolved'.

2.catch():

-It can accept a callback function

-The callback function given to catch() gets executed only when the promise retuns 'reject'

<----------*----------→

DOCUMENT:

-Document is an object created by browser

-The document object is the root node of DOM tree.

DOM:

-The every html element is considered as a node(js object) in DOM.

-DOM allows to modify the document content rendred by the browser without reloading the page.Therefore DOM helps to make a web page dynamic.

NOTE:

-Any modification done using DOM is not updated to the original page. Therefore , once we reload apage all the modification done using DOM will be lost.

-We can write the content on the browsers dynamically with the help of write and writeln method of document object.

EX:

To display a message on the browser page from javascript code:

```
document.write("hi")
```

-DOM is an API provided by the browser

-DOM is not javascript,it is built using javascript

-DOM is a object oriented represetation of html file.

-Every time an html document is given to the browser ,it automatically generates dom tree it can be accessed and modified using the root node document in javascript

PROPORTIES OF DOM OBJECT WHICH RETURNS DOM OBJECT REFERENCE:

1.firstChild

-It is a property, which holds the reference of first child node of the target node.

-syntax:

target_node. firstChild

-firstChild can be anything either a text or a comment or an element

2.firstElementChild

-It is a property ,it gives the reference of the element which is the firstchild of the target node.

-syntax:

target_node.firstElementChild

3.lastChild

-It is a property,it gives the reference of last child object of the target node.

-syntax:

target_node.lastChild

-It can either return text,comment or element object.

4.lastElementChild

-It is a property, it returns the reference of an element which is a last element of the target node.

5.children

-It is a property,it contains list of all the elements which are children

of target_node

6.childNodes

-It will give all the child nodes including text and comments

7.nextSibling

-It gives the reference of node which is immediate right sibling of DOM

8.nextElementSibling

-It gives the reference of the node which is next to target_node to its right.

9.parentElement

-It will just give the parent element of the current target_node

---

manipulation of dom

-->modifying and updating dom tree

a)adding elements to the dom

--->we can add elements in the dom in the following ways:

1)using the property in a InnerHTML.

2)by creating an element using CreateElement() method of dom object.

---

1)InnerHTML

--->innerhtml is a property of an dom element.

--->the innerhtml conatins everything between the tag as a string

--->we can update property innerhtml

synatx to update:-

target_elemet.innerHTML = "STRING"

target_element.InnerHTML += "string/with htmltags" (gives previous and updated tags)

disadventage:-

1)security issues :-(if i pass " " without giving += so, the white-space will be updated)

2)it will reduces the efficiency of browser,because the dom is recreated by every time innerHTML is modified.

---

2)using createElement() method

we have two steps :-

step 1 :- create an element object

syntax:-

document.createElement ('tagname');

step2 :- insert the element into the dom.

syntax:-

target_node.appendchild(reference_of_ele_object_to_be_added)

<----------*---------->

EVENT:

-Event is an action performed by a end user on a web page.

-A end user can perform different events such as -

1. Mouse events

2. Keyboard events et

two ways of adding events on dom:-

-->using jquery

-->using js

EVENT LISTENER:

-Event listener/handler is a function which executes task when an event is occured on the most specific element.

Event flows:

-Event capturing

-Event bubbling

Event capturing:

-In event capturing,an event starts from the top of the DOM tree(least specific element)& flows downwards upto the target element(most specific element).

eg:-

<html>

<head>

```
</head>
<body>

 <div>
 <input type="button" value="">
 </div>

</body>
```

Event bubbling:

-In event bubbling, the event starts at the target element(most specific element) and then flows in the upward direction of the dom tree

eg:-

```
<html>
<head>
</head>
<body>
 <div>
 <input type="button" value="">
 </div>

</body>
```

EVENT PROPAGATION:

-Event propagation has 3 phases:

1.Phase I:Event capturing occurs,it provides the oppurtunity to intercept the elements

2.Phase II:Actual target element recieves the event

3.Phase III:Event bubbling occurs,it provides an oppurtunity to have a final response to the event

eg:-

```
<html>
<head>
</head>
<body>

 <div>
 <input type="button" value="">
 </div>

</body>
</html>
```

TO ADD AN EVENT HANDLER TO AN ELEMENT:

We can add an event handler to an element with the help of addEventListener() method of DOM object.

Syntax:

target_element.addEventListener(a1,a2,a3);

a1 ---> event as a string

a2 ---> event handler

a3 ---> boolean value

NESTED EVENT:

-If there is as an event designed with the same event inside the nested elements then it is known as nested events.

EX:

Let us consider nested div tags where each div tag is having an event handler "click"

case 1:

If event "click" occurs on the grandParent then the event handler of grandParent is executed.

case 2:

when event occurs on the parent,

-the target is parent

-all event handlers of both parent and grandParent gets executed

-If we use event capturing flow,then event handlers from the top to target gets executed,therefore the output will be "grandParent clicked" and "parent clicked"

-If event bubbling flow is used,then the execution of event handler from target (bottom) to top.Therefore the output will be ,"parent clicked" then "grandParent clicked

for...in is used to iterate over all enumerable properties of an object, including inherited enumerable properties. This iteration statement can be used with arrays strings or plain objects, but not with Map or Set objects.

for...of is used to iterate over iterable objects, iterating over their values instead of their properties. This iteration statement can be used with arrays, strings, Map or Set objects, but not with plain objects.

forEach() is a method of the Array prototype, which allows you to iterate over the elements of an array. While forEach() only iterates over arrays, it can access both the value and the index of each element while iterating.

----------*----------

JavaScript JSON

----The JavaScript Object Notation (JSON) is a notable format for representing objects and values.

---JavaScript provides two methods of converting: JSON.stringify and JSON.parse. You can use the first one for converting objects into JSON and the second one- for converting JSON back into an object.

JSON.stringify

Let’s consider the following example of the JSON.

stringify usage:

```
let site = {
  name: 'W3Docs',
  hasAdmin: true,
  books: ['html', 'css', 'js', 'git'],
  booksCount: 4,
};
let json = JSON.stringify(site);
console.log(typeof json);
```

JSON.parse

For decoding a JSON-string another method is used, called JSON.parse

```
let value = JSON.parse(json);
```

----------*----------

Decision statements:

Decision statements helps to skip a block of instructions,when we dont have a favouring situation

Ex:The instructions of loading home page should be skipped if entered password is incorrect.

Note:

-Decision statements of JS:

1.If :

syntax:

if(condition)----any expression whose result is boolean

{

statements

}

2.if-else

syntax:

if(condition)

{

statements

}

else

{

statements

3.else-if ladder:

syntax:

if(condition 1)

{

statements

}

else if(condition 2)

```
{
statements
}
else if(condition 3)
{
statements
}
else if(condition 4)
{
statements
}
.
.
.
else
{
statements----------not a mandatory block.
}
```

SWITCH statement:

syntax:

```
switch(value)
{
case (value) : {console.log(statement)};
```

```
}
```

Ex:

```
switch(1)
{
 case 1 : {console.log('case 1')};
 case 2 : {console.log('case 2')};
 case 3 : {console.log('case 3')};
 default: {console.log('default')};
}
```

---

loop statements:(iterations):

The process of executing statement or block of statement repeatadly multiple times is known as loop.

Note:

When we design a loop it is the responsibility of programmer to break the loop after achieving the desired task.If not ,the program gets into infinite loop state.

Loop statements:while,do-while,for,for-in,for-of,for-each

while loop:

A while loop evaluates the condition inside the parenthesis ().

If the condition evaluates to true, the code inside the while loop is executed.

The condition is evaluated again.

This process continues until the condition is false.

When the condition evaluates to false, the loop stops.

syntax:

```
while(condition)
{
 statements to be repeated ;
}
```

Do-while loop:

The body of the loop is executed at first. Then the condition is evaluated.

If the condition evaluates to true, the body of the loop inside the do statement is executed again.

The condition is evaluated once again.

If the condition evaluates to true, the body of the loop inside the do statement is executed again.

This process continues until the condition evaluates to false. Then the loop stops.

syntax:

while

```
{
 statements to be repeated ;
}(condition)
```

---

for loop:

syntax:

```
for(intialization;condition;update)
{
 statements;
}
```

OBJECT:

-Any substance which has its existance in the real world is known as OBJECT.

-Every object will have properties(attributes of feeds which describe them).

-Every object will have actions(behaviours)

EX:

1.car:

properties of car: model,color,price

actions of car:accelarate,brake,start,stop

2.webpage:

properties of webpage:url,data/content on webpage.

actions of webpage:scroll,hover,click

Note:

-In programming an object is a block of memory which represents a real world.

-The properties of real world object are represented using variables

-The actions of real world object are represented using METHODS(FUNCTIONS).

-In javascript object is a collection of attributes and actions in the form

of key&value pairs enclosed

within curly braces{}

-Key represents properties/attributes of an object

-Value represents state of an object.

-In javascript,we can create js object in 3 different ways:

1.Object literal{}

2.Using a function

Note:

-If the key is not present in the object then we get 'undefined'

as the value.

-The objects in javascript are mutable(state of the object can be modified)

in nature.

task on object:

Task:

1.Let us consider a real world object marker with the following attributes

color,brand,price their states of black,camlin,25.

a) create javascript object to store.

b)Display the entire object.

c)log price

d)change black to blue and log it

e)add extra attribute shape into the marker object with value as cylinder

f)remove the attribute brand g)display the final object

task on array:

1.create an array with following elements: 10,30,40 & displpay the elements in the reverse order

2.Write an array to store names of atleast 3 cities,display all the cities whose length is even

----------*----------

Object Methods:

Object.create()

This method is used to create a new object with the specified prototype object and properties.

Object.entries()

This method returns an array with arrays of the key, value pairs.

Object.freeze()

This method prevents existing properties from being removed.

Object.keys()

This method returns an array of a given object's own property names.

Object.seal()

This method prevents new properties from being added and marks all existing

properties as non-configurable.

Object.values()

This method returns an array of values.

----------*----------

Local storage has 4 methods:

setItem() Method – This method takes two parameters one is key and another one is value. It is used to store the value in a particular location with the name of the key.

localStorage.setItem(key, value)

getItem() Method – This method takes one parameter that is key which is used to get the value stored with a particular key name.

localStorage.getItem(key)

removeItem() Method – This is method is used to remove the value stored in the memory in reference to key.

localStorage.removeItem(key)

clear() Method – This method is used to clear all the values stored in localstorage.

localStorage.clear()

Session Storage has 4 methods:

setItem() Method – This method takes two parameters one is key and another one is value. It is used to store the value in a particular location with the name of the key.

sessionStorage.setItem(key, value)

getIteam() Method – This method takes one parameter that is key which is used to get the value stored with a particular key name.

sessionStorage.getItem(key)

removeItem() Method – This is method is used to remove the value stored in the memory in reference to key.

sessionStorage.removeItem(key)

clear() Method – This method is used to clear all the values stored in the session storage

sessionStorage.clear()

Cookie: The term "cookie" refers to just the textual information

about a website.

Difference Between Local Storage, Session Storage, And Cookies

Local Storage:

---The storage capacity of local storage is 5MB/10MB

---As it is not session-based, it must be deleted via javascript or manually

---The client can only read local storage

---There is no transfer of data to the server

---There are fewer old browsers that support it

Session Storage:

---The storage capacity of session storage is 5MB

---It's session-based and works per window or tab. This means that data is stored only for the duration of a session, i.e., until the browser (or tab) is closed

---The client can only read local storage

---There is no transfer of data to the server

---There are fewer old browsers that support it

Cookies:

---The storage capacity of Cookies is 4KB

---Cookies expire based on the setting and working per tab and window

---Both clients and servers can read and write the cookies

---Data transfer to the server is exist

---It is supported by all the browser including older browser

----------*----------

Javascript String methods:

Some of the important javascript string functions include:

charAt(x)

charCodeAt(x)

concat(v1,v2..)

fromCharcode(c1,c2)

indexOf(substr, [start])

lastIndexOf(substr, [start])

replace(regexp/substr, replacetext)

search(regexp)

slice(start, [end])

split(delimiter, [limit])

substr(start, [length])

substring(from, [to])

toLowerCase()

toUpperCase()

includes()

endsWith()

repeat()

valueOf()

trim()

1.charAt(x)

This function will return the character at the x position within the string.

```
var myString = 'jQuery FTW!!!';
console.log(myString.charAt(7));
//output: F
```

2.charCodeAt(x)

This function will return the unicode value of the character at position ‘x’ within the string.

3.concat(v1,v2..)

This function combines one or more strings(argv1,v2 etc) into existing one

4.slice(start, [end])

This function returns a substring of the string based on the “start” and “end” index arguments, NOT including the “end” index itself. “End” is optional, and if none is specified, the slice includes all characters from “start” to end of the string.

```
//slice(start, end)
var text="excellent"
text.slice(0,4) //returns "exce"
text.slice(2,4) //returns "ce"
```

5.split(delimiter, [limit])

This will split a string into many according to the specified delimiter, and returns an array containing each element.

6.toLowerCase()

This will return the string with all of its characters converted to lowercase.

```
var myString = 'JAVASCRIPT ROX';
myString = myString.toLowerCase();
console.log(myString)
//output: javascript rox
```

toUpperCase()

This will return the string with all of its characters converted to uppercase.

```
var myString = 'javascript rox';
myString = myString.toUpperCase();
console.log(myString)
//output: JAVASCRIPT ROX
```

includes()

It is used to check whether a string contains the specified string or characters.

```
var mystring = "Hello, welcome to edureka";
var n = mystring.includes("edureka");
//output: True
```

endsWith()

This function checks whether a string ends with

specified string or characters.

```
var mystr = "List of javascript functions";
var n = mystr.endsWith("functions");
//output: True
```

repeat()

This returns a new string with a specified number of copies of an existing string.

```
var string = "Welcome to Edureka";
string.repeat(2);
//output: Welcome to Edureka Welcome to Edureka
```

trim()

This function removes whitespace from both ends of a string.

```
var str = "     Hello Edureka!     ";
alert(str.trim());
```

substring() is similar to slice().

// The difference is that substring() cannot accept negative indexes.

// substr() is similar to slice(). and substr() is deprecated and will not accept negative value

```
-----------------------------------------------------

// let str = 'name'
// console.log(str) /// created a string using literals

// //--------------------

// // using new keyword

// let str1 = new String('foo')
// console.log(str1)

// -----------------------------------


///string methods /////

// lenght
var str=`hello`;
var n=str.length;
console.log(n);//5

// slice
//fetch the part of string and returns new string
//strig.slice(start , end) leaves end count and accepts negative values
var str1 = "javascript";
var sc = str1.slice(0, 4);
```

```
console.log(sc); //java

// //or using negative values

var str1='java script';
var sc=str1.slice(-10);
console.log(sc);//ava scrip

// touppercase
var str2=`javascript`;
var up=str2.toUpperCase();
console.log(up);//JAVASCRIPT

// // tolowercase
var str3=`JAVASCRIPT`;
var low=str3.toLowerCase();
console.log(low);//javascript

//  concat two add two strings
var str4=`Mango`;
var str5=`DB`;
console.log(str4.concat(str5));//MangoDB

// // charAt
var str6=`sheela`;
var char=str6.charAt(3);
console.log(char);//e
```

```
//charCodeAt
var str7=`sheela`;
var charcd=str7.charCodeAt(5);
console.log(charcd);//97

//repeat
var str8 = `javascript `;
var rpt = str8.repeat(5);
console.log(rpt); //javascriptjavascriptjavascriptjavascriptjavascript

// replace
var str9=`pune to bangolore`;
var rplc=str9.replace(`pune`,`mangalore`);
console.log(rplc);//mangalore to bangolore

//trim
var str10=` javascript `;
var trm=str10.trim();
console.log(trm);//javascript(remove white space from both sides of string)

// startsWith()
var h = "JavaScript Is case Senitive";
var k = h.startsWith("j");
console.log(k); // false

// endsWith()
```

```
var h = "Working with Js Is Awesome";
var k = h.endsWith("e");
console.log(k); // true

// split eg 1
var str11=`Aladin joker`;
var splt=str11.split(``);
console.log(splt);//(12) ['A', 'l', 'a', 'd', 'i', 'n', ' ', 'j', 'o', 'k', 'e', 'r']

// split eg2
var str11=`Aladin joker`;
var splt=str11.split(` `);
console.log(splt);//(2) ['Aladin', 'joker']

// split eg3
var str11=`Aladin joker`;
var splt=str11.split(`| `);
console.log(splt);//['Aladin joker']

// substring
//syntax: substring(start, end);
var str12=`good night `;
var substr=str12.substring(7,1);
console.log(substr);//ood ni

//substr
```

```
// The JavaScript string substr() method fetch the part of the given string and return the new string.
var str13=`javascript`;
var sbstr=str13.substr(0,4);
console.log(sbstr);//java

//indexof

var s = "India is one of powerful country";
var s1 = s.indexOf("i"); // take in index format starts with zero
console.log(s1); //3

// includes()
var e = "JAVAjavaScript Are Not Same";
var f = e.includes("Script");
console.log(f);//true

// note:
// substring() is similar to slice().
// The difference is that substring() cannot accept negative indexes.

// note:

// substr() is similar to slice(). and substr() is  deprecated and will not accept  negative value
```

----------*----------

The JavaScript string is a sequence of characters.

There are 2 ways to create string in JavaScript

By string literal

By string object (using new keyword)

1) By string literal

The string literal is created using double quotes.

```
var stringname="string value";
```

2) By string object (using new keyword)

The syntax of creating string object using new keyword is given below:

```
var stringname=new String("string literal");
```

Here, new keyword is used to create instance of string.

----------*----------

String methods:

toLowerCase()

The toLowerCase() method returns the calling string value

converted to lowercase.

concat()

The concat() method concatenates the string arguments to the calling string

to return a new string.

indexOf()

The indexOf() method searches the entire calling string and returns the index of the first occurrence of the specified substring.

endsWith()

The endsWith() method determines whether a string ends with the characters of a specified string.

repeat()

The repeat() method constructs and returns a new string that contains the specified number of copies of the string on which it was called.

replace()

The replace() method returns a new string with one, some, or all matches of a pattern that will be replaced by a replacement.

slice()

The slice() method extracts a section of a string and returns it as a new string without modifying the original string.

split()

The split() method takes a pattern and divides a String into an ordered list of substrings by searching for the pattern,

puts these substrings into an array, and then returns it.

toString()

The toString() method of a string object returns a string representing the specified string.

----------*----------

VARIABLES:

a named block of memory which is used to store a value is known as variables

(container to store data).

Note:

-in js variables are not strictly typed,it is dynamically typed.

Therefore it is not necessary to specify type

of data during variable declarations.

-in a variable we can store any type of value.

-it is mandatory to declare a variable before using.

-syntax: var/let/const identifier;

ex:

```
var a ;//declaration
a = 10;//initialization or assignment
console.log(a);//10
let b ;
b = 10;
console.log(b);//10
```

```
const c = 30 ;
console.log(c);//30
```

Note:

when a variable is declared in js,js engine implicitly assigns undefined value to it.

ex:

```
var a;
console.log(a);//undefined
```

UNDERSTANDING EXECUTION IN JS:

1.everytime when js engine runs a js code,it will 1st create a 'global execution context'.

2.The Global Execution context has 2 parts:

a.variable area

b.functional/executional area

3.JS engine generally uses 2 phases to execute a JS code

--> phase 1:all the memory is allocated for the declarations in top to bottom order and assigned with the default value 'undefined' in variable area of global execution context.

-->phase 2:all the instructions get executed in the top to bottom order in execution area of global execution context.

HOISTING: JS allows a programmer to use a member (variable) before the

declaration statement.This characteristic is known as Hoisting.

DIFFRENCES BETWEEN var let & const:

var:

1.var is global scoped

2.we can declare multiple variables with the same name(most recently created variable is used).

3.The value of variable can be modified.

4.Variables declared using var is hoisted,does not belong to temporal dead zone.Can be used before initialization.

```
Ex:console.log(a);//undefined
var(a);
```

Ex:

```
var a = 10;
a =20;
console.log(a);//20
```

let:

1.let is local scoped

2.we can not declare two variables with the same name within a block.

3.The value of variable can be modified.

4.Variables declared using let is hoisted,and it does belong to temporal dead zone.Can not be used before initialization.

```
Ex:console.log(a);//reference error(because of TEMPORAL DEAD ZONE)
var(a);
```

ex:

```
let a = 10;
a =20;
console.log(a);//20
```

const:

1.it is local scoped

2.we can not declare two variables with the name within a block.

3.The value of variable can not be modified.

4.Variables declared using let is hoisted,and it does belong to temporal dead zone.Can not be used before initialization.Ex:console.log(a);//reference error (because of TEMPORAL DEAD ZONE)

```
var(a);
```

ex:

```
const a = 10;
a =20;//typeError
console.log(a);//error,assign the value only once.
```

TEMPORAL DEAD ZONE:

The time between the variables which are created and its intialization is known as TDZ.

-in TDZ period we can not use variables.

-let and const belong to TDZ

Ex:

let a;

clg(a);//undefined

a=10;

-the problem occurs only if the variables used before declaration.

clg(a);//reference error (TDZ)

let a;

a=10;

----

type-casting:

The process of converting one type of data to another type of data.

implict type-casting

the process of converting one type of data into another type of data by JS engine implicitly(implicit typecasting),when a wrong type of data is used in the expression is known as type coercion(or implicit type casting).

EX:

console.log(10+'20');//1020,number is coverted to string

console.log(10-'2');//8,string is coverted to number

console.log(5-'a');//NaN, the string does not a have valid numeric value, hence it is converted to a special number 'NaN'.

What is Nan?

NaN(Not a Number) is number!!!

What is Nan?

NaN(Not a Number) is number!!!

Any arithmetic operation with NaN, result is NaN

EXPLICIT TYPECASTING:

The process of converting one type of value to another type of value is known as explicit typecasting.

Conversion of any number to number:

1.String to number:

if a string is valid number,we get the real number.

if the string consist any other character,then we get NaN as output.

EX:

```
console.log(typeof 'a');
console.log(Number('a'))//NaN
console.log('123'+10);//12310
console.log(Number('123')+10);//133,string '123' is converted to number
explicitly(by programmer)
console.log(Number('123a'+10));//NaN
```

----------*----------

Z-Index:

---->Z Index (z-index) is a CSS property that defines the order of overlapping HTML elements.

Elements with a higher index will be placed on top of elements

with a lower index.

----> it will have a default value of auto

Opacity Property:

--->it is used to provide blurness(transparency) for element

---->The opacity property takes values from 0.0 to 1.0, with 1 being the default value for all elements.

The lower the value, the more transparent. So if an element is given an opacity of 0, it would be invisible.

---->An opacity value of 1 is the default

filter Property:

--->The filter CSS property applies graphical effects like blur or color shift to an element. Filters are commonly

used to adjust the rendering of images, backgrounds, and borders.

--->some of filter property are:

filter: blur(pixels)

filter: brightness(percentage)

filter: contrast(percentage)

filter: drop-shadow(x-axis y-axis blur color_name)

filter: grayscale(percentage)

filter: hue-rotate(degree)

filter: invert(percentage)

filter: opacity(percentage)

filter: saturate(percentage)

----------*----------